Departamento de Informática, Univ. de Valencia

Sistemas Inteligentes

Juan Domingo
Juan.Domingo@uv.es

Septiembre de 2026

Tema 2:

Búsqueda

Índice del tema 2: comienzo de la sección 1

1 Espacios de estados
1.1 Definiciones
1.2 Arboles de búsqueda

2 Estrategias de búsqueda ciega
2.1 Definición y algoritmos

3 Estrategias de búsqueda informada o heurística
3.1 Funciones heurísticas
3.2 Algoritmo A^*
3.3 Variantes del A^*PI
3.4 Otros algoritmos

Definición de estado

Estado

Dado un sistema físico o lógico (virtual) suficientemente complejo (compuesto de varias partes que se relacionan entre sí), nos interesa analizar su evolución temporal cuando un agente realiza actuaciones en él.
En principio, un sistema viene descrito por muchos parámetros. De ellos, elegiremos algunos que consideramos relevantes para nuestro propósito, que normalmente es llevarlo a una situación concreta.

Estado de un sistema es el conjunto de valores de aquellos parámetros relevantes que describen de modo unívoco la situación actual del sistema, y son suficientes para determinar qué acciones pueden realizarse sobre él, y cómo cada una de esas acciones cambiará estos valores (es decir, cambiará el propio estado).

Ejemplos de estados

En el juego del ajedrez, y con el propósito de jugar contra un ordenador, un estado puede definirse dando la posición que ocupa cada pieza.

Otras variables, como el material, peso y forma de las piezas, no serían relevantes aquí, pero sí lo serían si se pretendiese que un robot moviera físicamente las piezas.
Nótese que el estado podría representarse como un array de 8×8 del valores de un par (P,C) donde P es del tipo definido por enumeración P={peón, torre, álfil, caballo, reina, rey, vacío} y C del tipo C={blanco,negro}, o con una lista de dos listas (piezas blancas y negras, cada una de tripletes de la forma (P,fila,columna)) o de otras maneras. La estructura de datos que escojamos no debe confundirse con el estado.

En un sistema de predicción bursátil de un mercado dado (p. ej. la bolsa de Madrid), un estado sería la expresión de la cotización actual de cada valor (compañía cotizada) y de los valores de parámetros relevantes como la tasa de paro, la cotización de ciertas monedas, el índice de otras bolsas, etc.

Este ejemplo muestra que no siempre es claro cuál es la elección de variables que serán relevantes para nuestro propósito, sobre todo en sistemas sometidos a incertidumbre o influídos por factores humanos y/o desconocidos.

Definición de espacio de estados

Espacio de estados

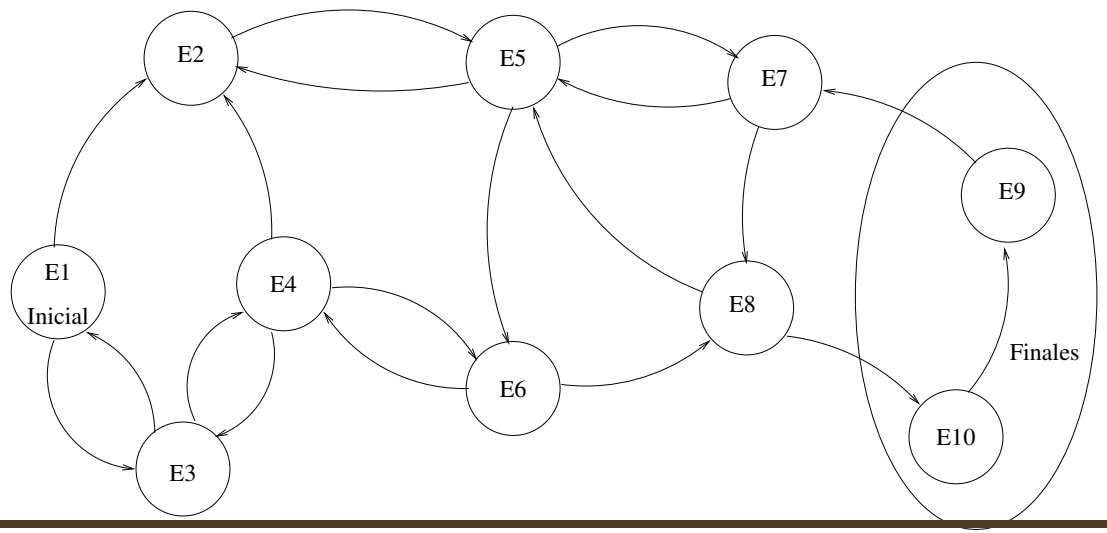
Es el conjunto de todos los estados posibles para el sistema, junto con una descripción que indique a cuáles de ellos puede el sistema evolucionar desde cualquier estado dado. Supondremos que:

- Las variables que describen el estado están **cuantizadas**, es decir, sólo toman valores en un conjunto discreto.
- Los conjuntos de valores para cada variable pueden ser finitos (con lo que el número de estados posible será finito, aunque quizás enorme) o infinitos (con lo que el número de estados es directamente infinito).
- Hay un conjunto conocido de reglas que dicen qué estados son válidos.
- El sistema evoluciona en instantes de tiempo discretos.
- Hay un conjunto conocido de reglas que dicen a qué estados puede llegar el sistema en el siguiente instante de tiempo desde cualquier estado dado. Cada una de esas reglas se llama **operador**.
- Se conoce cuál es el estado inicial del sistema y cuál es el estado(s) que se considera(n) meta. Hay reglas para saber si un estado es meta o no (incluso su simple inclusión en una lista).

Representación del espacio de estados

La forma de representar cada estado se verá posteriormente, pero la forma de representar el sistema completo y su evolución es casi siempre en forma de **grafo dirigido**: cada posible estado es un nodo, y existe arco entre dos nodos si hay una regla que dice que se puede pasar de uno a otro en un sólo instante de tiempo discreto (equivalentemente: si se puede aplicar algún operador que lleva de un estado al otro). Por tanto, cada nodo está etiquetado con un estado y cada arco, con un operador.

Además, cada arco puede tener asociado un **coste**, que es un número real (normalmente positivo) que indica el esfuerzo o los recursos que supone aplicar ese operador al estado del nodo origen.



Definición de algoritmo de búsqueda

Es el algoritmo que, conociendo...

- bien
  el grafo de estados completo
  o bien
  las reglas de determinación de la validez de un estado y
  las reglas de posibilidad de aplicación de los operadores
- bien
  el estado o conjunto de estados finales (metas)
  o bien
  reglas para determinar si un estado es meta,

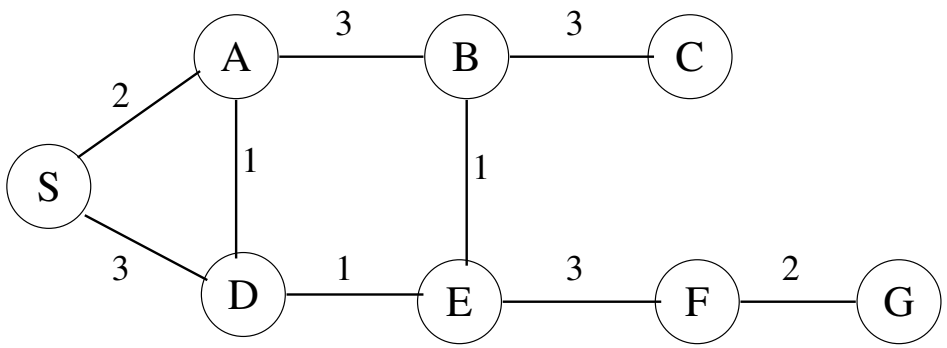
toma como entrada el estado inicial y devuelve como salida una lista finita ordenada de ternas operador/estado/coste,
$L = \{(Op_1, E_1, C_1), (Op_2, E_2, C_2), ..., (Op_n, E_n, C_n)\}$ donde
$E_1 = Op_1(E_{ini})$, $E_i = Op_i(E_{i-1})$, $C_i = Coste(Op_i(E_{i-1}))$ y
el último estado E_n está en el conjunto de estados meta.

Arbol de búsqueda (I)

Arbol de búsqueda

Es una estructura de datos árbol en que se representa como nodo raíz el estado inicial y como nodos hijos de uno dado, todos los estados a los que se puede llegar desde el nodo padre por aplicación de cada operador válido.

A diferencia del grafo de estados, un árbol por su propia naturaleza no es cíclico (aunque será infinito, si resulta de un grafo cíclico); en él, un mismo estado puede aparecer como nodo muchas (o incluso infinitas) veces. No obstante, un árbol siempre puede representarse como árbol finito, si se omite cualquier nodo que ya se encuentre entre sus propios antecesores. Esto evita que un algoritmo de búsqueda quede atrapado en un bucle infinito.
Ejercicio: transformar el grafo no dirigido de la figura en un árbol finito.
(recuérdese que un grafo no dirigido puede verse como uno dirigido, con cada arco resumiendo otros dos).



Arbol de búsqueda (II)

Ejercicio: intentar determinar el grafo de estados y al menos una pequeña parte del árbol de búsqueda del tres en raya (asumiendo que las fichas son `x` y `o`, y que comienza el juego la `x`).

Estrategia de búsqueda

La regla o conjunto de ellas que determina cuál de los nodos descendientes del nodo actualmente explorado debe ser el siguiente a explorar.

Problemas abordables

Los problemas tratables con esta formulación pueden ser de tres tipos:

- Problemas para los que existe un algoritmo que da la solución en tiempo finito, y practicable (al menos, polinómico con el tamaño del problema). Ej.: la oveja, el lobo y la col, las torres de Hanoi,...
- Problemas para los que existe un algoritmo de solución en tiempo finito, pero es computacionalmente intratable (no es polinómico, o el árbol de búsqueda es inmenso). Ej.: ajedrez, go,...
- Problemas para los que no existe un algoritmo que garantice encontrar la solución en tiempo finito (básicamente, porque el espacio de estados es infinito). Ej.: la demostración de teoremas en lógica formal de primer orden.

Las técnicas de IA se ocupan de encontrar soluciones a los dos últimos casos en un tiempo asumible, aun cuando no se trate necesariamente de la solución óptima.

Índice del tema 2: comienzo de la sección 2

1 Espacios de estados
1.1 Definiciones
1.2 Arboles de búsqueda

2 Estrategias de búsqueda ciega
2.1 Definición y algoritmos

3 Estrategias de búsqueda informada o heurística
3.1 Funciones heurísticas
3.2 Algoritmo A^*
3.3 Variantes del A^*PI
3.4 Otros algoritmos

Definición

Búsqueda ciega (no informada)

Es aquella en la que la elección del siguiente nodo del árbol de búsqueda no usa más información que la dada por la estructura del propio árbol; como máximo, tiene en cuenta el coste de los arcos a los nodos descendientes.

Un algoritmo de búsqueda ciega es independiente de la naturaleza del problema, y se aplica por igual sea cual sea el contenido de los nodos o la forma del grafo de estados.

Búsqueda preferente por amplitud

Consiste en expandir el nodo inicial, y luego sus sucesores, y luego los sucesores de los sucesores, hasta alcanzar un nodo meta.

Si d es la profundidad de la solución que está a menor profundidad y b el factor de ramificacion, el coste computacional [(1)], tanto espacial como temporal, es
$b + b^2 + ... + b^d = \mathcal{O}(b^d)$

Esta búsqueda es completa (siempre se encuentra el nodo meta), y es óptima en el sentido de que encuentra el camino a un nodo meta con el menor número de acciones posible (es decir, el que llega recorriendo el menor número de arcos desde el nodo inicial), aunque puede no ser el de menor coste de recorrido.

[(1)] Atención: no confundir coste computacional con el coste de recorrido, el asociado a los arcos

Búsqueda de coste uniforme

Consiste en expandir los nodos moviéndose siempre al que va a dar un menor coste de arco acumulado desde el nodo inicial, hasta alcanzar un nodo meta.

El coste computacional de la búsqueda, tanto espacial como temporal, es
$b + b^2 + \ldots + b^d = \mathcal{O}(b^d)$

Esta búsqueda encuentra el camino de menor coste de arco a un nodo meta (siempre que el coste de cada arco sea positivo o nulo). Coincide con la anterior si todos los arcos tienen asociado el mismo coste.

Búsqueda preferente por profundidad

Consiste en expandir el primer sucesor de cada nodo, y después el primero de éste, etc. hasta llegar bien a un nodo meta, o bien al final del árbol (nivel más profundo). En este segundo caso, se asciende de nivel y se repite el procedimiento con el segundo nodo, y así hasta que no queden.

Si m es la máxima profundidad de árbol, el coste computacional espacial es $\mathcal{O}(bm)$ (minimiza la memoria necesaria) y el temporal es como antes, $\mathcal{O}(b^d)$.

En caso de espacios de estados infinitos puede quedar estancado en bucle infinito, salvo que se lleve lista de los estados visitados, lo cual incrementa el coste computacional espacial. El coste de arco no se usa, pero se puede tener en cuenta ordenando por coste de arco de menor a mayor la lista de sucesores de cada nodo.

Búsqueda limitada por profundidad

Esta búsqueda procede igual que el caso anterior (preferente por profundidad) pero se para bien al llegar al nodo meta, bien al llegar a un límite de profundidad fijado, sea l (o a nivel más profundo, si esa rama tuviera profundidad menor que l).

Si l es el límite de profundidad establecido, el coste computacional espacial es $\mathcal{O}(bl)$ (minimiza la memoria necesaria) y el temporal es $\mathcal{O}(b^l)$

Esta estrategia no puede caer en bucle infinito, pero no es completa (podría no haber ninguna meta hasta alcanzable a profundidad l) y no es óptima (aun habiéndola, podría haber otra con menor coste de arco a profundidad mayor).

Búsqueda por profundización iterativa

Siendo m la profundidad máxima del árbol, se realizan una tras otra m búsquedas limitadas en profundidad con límites $l = 1, 2, ..., m$.

Obviamente, los costes computacionales espaciales y temporales son como en la búsqueda preferente por profundidad, $\mathcal{O}(bm)$ (minimiza la memoria necesaria) y $\mathcal{O}(b^d)$.

Pese a que pueda parecer extraña, dado que cada iteración está repitiendo parte de la búsqueda llevada a cabo en la anterior (se expanden muchos de los mismos nodos), tiene las ventajas de no poder caer en bucle infinito, emplear muy poca memoria ($\mathcal{O}(bd)$) y ser completa (si existe solución, se encuentra, y es la mejor en términos de coste de arcos). Por ello, es una de las estrategias ciegas más usadas.

Búsqueda bidireccional

Procede reaizando dos búsquedas en amplitud, una comenzando en el nodo origen y otra en uno de los nodos meta (o varias en cada uno de los nodos meta, si los hubiera). El algoritmo acaba cuando ambas se encuentran en un nodo común.

Los costes computacionales espacial y temporal son como en la búsqueda por amplitud, pero con la profundidad máxima reducida a la mitad (cada búsqueda tiene que profundizar/ascender como máximo $d/2$ niveles), siendo por tanto $\mathcal{O}(b^{d/2})$.

Al igual que la búsqueda en amplitud, es una estrategia completa y óptima.

Índice del tema 2: comienzo de la sección 3

1 Espacios de estados
1.1 Definiciones
1.2 Arboles de búsqueda

2 Estrategias de búsqueda ciega
2.1 Definición y algoritmos

3 Estrategias de búsqueda informada o heurística
3.1 Funciones heurísticas
3.2 Algoritmo A^*
3.3 Variantes del A^*PI
3.4 Otros algoritmos

Búsqueda heurística: Definición

Búsqueda heurística (informada)

Es aquella que utiliza conocimiento "inyectado" en el algoritmo por el programador, en forma de reglas o de función de preferencia que determine el siguiente estado al que moverse, cuando no existe una manera de determinarlo porque no se puede explorar el árbol de estados completo, bien sea debido a su tamaño, o más raramente, porque cambia con el tiempo. Una buena heurística debe cumplir cuatro propiedades:

- El esfuerzo computacional para encontrar una solución aceptable debe ser asumible.
- La solución encontrada debe ser, en promedio, próxima a la óptima.
- La probabilidad de encontrar una solución pobre (muy subóptima) debe ser baja.
- La heurística debe ser simple, fácil de explicar, entender y programar.

Normalmente una heurística se codifica en forma de función $f(n)$ que toma un nodo n y devuelve un valor **positivo o nulo** tanto menor cuanto más cerca esté n de un nodo objetivo. (Cerca en términos de movimientos en el árbol de búsqueda)

Búsqueda preferente por lo mejor

Es una búsqueda en amplitud, pero modificada para tener en cuenta $f(n)$. Su algoritmo es:

1. Expandir el nodo inicial y almacenar en un conjunto L sus sucesores.
2. Si L contiene un estado meta, acabar.
3. Si no, expandir el nodo de L con menor valor de $f(n)$, sea n_0, añadir sus sucesores a L y quitar al propio n_0 de L.
4. Ir al paso 2

Obviamente, esta búsqueda será óptima si lo es la función de evaluación, lo que no es el caso general. Tiene el problema de que no se tiene en cuenta el costo del camino recorrido hasta cada nodo a expandir.

Búsqueda A^*: notación

Sea N el conjunto de nodos del árbol de búsqueda.
Sea n_i el nodo inicial y sea $n \in N$ un nodo cualquiera.
Definamos una función $f : N \longrightarrow \mathbb{R}^+$ como $f^*(n) = g(n) + h^*(n)$ donde:
$g(n)$ es el coste mínimo para llegar desde n_i hasta n
$h^*(n)$ es el coste mínimo para llegar desde n hasta el nodo objetivo más cercano, sea n_g

Tanto $g(n)$ como $h^*(n)$ deben ser números reales ≥ 0.
$g(n)$ es conocida, o al menos, lo irá siendo a medida que vayamos recorriendo el árbol. Sin embargo, $h^*(n)$ obviamente no lo es (¿cómo podríamos saber el coste del camino óptimo si aún no sabemos cuál es ese camino?).

Por tanto, el A^* tomará como función heurística una $f(n) = g(n) + h(n)$, donde $h(n)$ es lo que propiamente se conoce como heurística, y debe ser una estimación juiciosa de $h^*(n)$.
Por tanto, f es una **estimación** del coste total para llegar desde n_i hasta n_g, pasando por n.

Búsqueda A^*: heurísticas posibles

¿Cómo podemos construir $h(n)$?

Si no tenemos absolutamente ninguna información sobre el problema, puede tomarse $h(n) = 0 \forall n \in N$, la llamada **heurística no informativa**.

Si la tenemos, es extremadamente conveniente que construyamos $h(n)$ de modo que

$0 < h(n) \leq h^*(n) \forall n \in N$.

Cuando esto ocurre se dice que la heurística h es **admisible**

La pregunta obvia es ¿cómo podemos saber que h es admisible, si no comocemos h^*?. Efectivamente, no siempre es posible y debe razonarse para cada caso concreto.

Búsqueda A^*: algoritmo A

Global var	Type	Initial value
gScore	Map [Node $\rightarrow \mathbb{R}^+$]	gScore[start] $\leftarrow 0$ gScore[n] $\leftarrow \infty \forall n \neq$start
fScore	Map [Node $\rightarrow \mathbb{R}^+$]	fScore[start] $\leftarrow$ h(start) fScore[n] $\leftarrow \infty \forall n \neq$start

Defined functions:
$h : N \longrightarrow \mathbb{R}^+$
$h(n)$: heuristic value for node n
$d : N \times N \longrightarrow \mathbb{R}^+$
$d(n_1, n_2)$: arc weight between nodes n_1 and n_2, if neighbours; ∞ if not.

```
function Aalgorithm(start,goals)   // Parameters: start is a node; goals is a set of nodes
open ← {start}        // open is of type set of nodes. Initialized to the set with just 'start'
closed ← {}           // also a set of nodes, initialized to the empty set
While not empty(open)
   current ← node in open with lowest fScore
   If current ∈ goals
      return (success)
   open.Remove(current)       // current is not to be expanded any more
   closed.Add(current)        // mark current as visited
   for each neighbor nei of current
      If nei ∉ closed
         tentative_gScore ← gScore[current] + d(current,nei)
         If tentative_gScore < gScore[nei]      // This neighbour is the best, up to now...
            gScore[nei] ← tentative_gScore   // Update its g-value...
            fScore[nei] ← gScore[nei]+h(nei)   // ... and update also f, since it is g+h
            If nei ∉ open
               open.Add(nei)            // add this neighbour to those to be expanded
return (failure)
```

Búsqueda A^*: casos especiales

Si se toma como heurística h la no informativa ($h(n) = 0 \forall n \in N$) el algoritmo A se convierte en el de búsqueda preferente por lo mejor (expandir todos los descendientes, yendo primero a aquel al que menos nos cueste llegar desde aquí).

Si se toma como h una heurística que sea admisible, entonces al algoritmo A se le llama A^*.

Como quiera que $h^*(n) \geq 0$ por definición, la heurística no informativa es siempre admisible (obviamente, $h(n) = 0 \leq h^*(n)$).

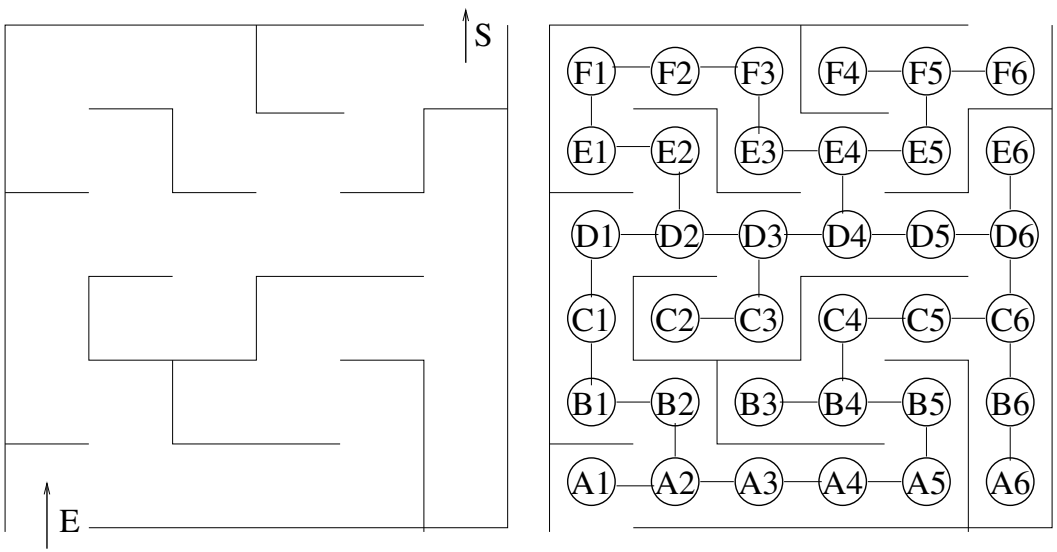
Dados dos algoritmos A^*, A_1^* y A_2^*, asociados a heurísticas (admisibles, claro)

$$h_1(n) \text{ y } h_2(n) \text{ con } h_2(n) \geq h_1(n) \forall n \in N$$

se dice que A_2^* es un algoritmo **más informado** que A_1^*. (Estando siempre por debajo del valor de h^*, puesto que es admisible, sin embargo se acerca más a este valor. Podríamos también decir "mejor" informado).

Ejemplo de A^*

Se resolverá en clase el ejemplo del siguiente laberinto, usando como $h(n)$ la heurística óptima, $h^*(n)$, que se puede ver en la tabla, y usando otra heurística admisible:



Valor de h^*

	1	2	3	4	5	6
A	12	11	12	13	12	9
B	9	10	11	10	11	8
C	8	7	6	9	8	7
D	7	6	5	4	5	6
E	8	7	4	3	2	7
F	7	6	5	2	1	0

Búsqueda A^*PI

Es una combinación de la técnica de búsqueda ciega por profundización iterativa, que ya vimos, con el algoritmo A^*.

De hecho, la idea es que se harán sucesivamente búsquedas A^*, parando cada una **no** cuando se llegue a un límite de profundidad, sino a un **límite del coste** dado por la función f. Cuando se han expandido todos los nodos cuya f sea mayor o igual que el límite actual, se aumenta éste y se comienza de nuevo.

El método es completo y óptimo, su coste computacional espacial es muy inferior al A^* y el temporal es superior (como en la búsqueda por profundización iterativa, se repite trabajo en cada iteración), aunque sin aumentar el órden de magnitud.

Búsqueda A^*SRM

Es una mejora de la anterior búsqueda (A^*PI). La mejora consiste en que, una vez efectuada una búsqueda A^* limitada por un valor de f, toda la memoria que reste hasta llenar la disponible se usa para almacenar los caminos que llevaron a los mejores nodos, ordenados de menor a mayor por valor de f. Esto hace que en la siguiente iteración no se tenga que repetir toda la búsqueda, sino que se puede comenzar desde el mejor nodo.

Este método es completo sólo si la memoria disponible es superior o igual al tamaño de la ruta hasta la meta más cercana.
Similarmente, es óptimo sólo si la memoria disponible es superior o igual al tamaño de la ruta hasta la solucion (meta) óptima más cercana.

Búsqueda por descenso (ascenso) de cima

Consiste simplemente en moverse siempre hacia el nodo sucesor que presente mejor (en nuestro caso, menor) valor de *h* (ATENCION: no de *f*), parando cuando se alcanza un nodo cuyos sucesores tienen todos un valor de *h* peor (en este caso, mayor) que el actual. Dicho nodo es, por tanto, un mínimo local. En otros contextos se le conoce también como método de descenso por gradiente.(1)

Es el método de mínimo coste, tanto espacial como temporal, pero no es completo (ni por supuesto óptimo), dado que, al no aceptar la posibilidad de volver atrás (volver a ascender el árbol de búsqueda para intentar otra rama) puede quedarse estancado en un mínimo local. Es, sin embargo, apropiado si podemos demostrar que el problema tiene un único mínimo de *h*, o si el mínimo encontrado, aun no siendo óptimo, es adecuado para nuestros propósitos (no hay una 'meta' sino que cualquier nodo suficientemente bueno es considerado aceptable).

---

(1): A veces se toma la función h contraria a nuestro criterio, de modo que se intenta maximizarla y el método se llama de ascenso por gradiente.

Búsqueda TABU

Búsqueda por ascenso (descenso) de cima con reinicio aleatorio

Es una mejora del anterior: lo ejecuta hasta llegar a un mínimo local, y vuelve a ejecutarlo varias veces, pero empezando cada vez desde un nodo distinto, elegido al azar. Al final nos quedamos con el mejor mínimo local obtenido (supuesto que ninguno de ellos sea meta).

Obviamente, sólo sirve para problemas en que el estado inicial no es importante (lo podemos elegir) sino sólo el estado final. Hay variantes muy sofisticadas que evitan ciertos grupos de estados, aun cuando parezcan mejorar *h* (de ahí el nombre), basándose en instancias de búsqueda anteriores.